Automated E-Learning Data Pipeline

User Manual

Audience: Data Engineers & Power BI Analysts

Version 1.0

1. Introduction

The Automated E-Learning Data Pipeline collects course information from Coursera, Udemy, and Udacity, then cleans and standardizes it into a single, query-ready Star Schema data warehouse. This manual explains how Data Engineers and Power BI Analysts operate the pipeline day to day, in both the Local (Docker/Airflow/DuckDB) and Cloud (Microsoft Fabric/Power BI) environments.

1.1 Who Should Use This Guide

- Data Engineers — responsible for running, monitoring, and troubleshooting the pipeline (scrapers, Airflow DAG or Fabric pipeline, dbt models).
- Power BI Analysts — responsible for connecting to the resulting star schema, refreshing semantic models, and building/consuming reports.

1.2 Two Deployment Modes

Mode	Orchestrator	Transform	Database	BI Layer
Local	Apache Airflow (Docker Compose)	dbt-duckdb	DuckDB (single file)	Manual SQL / local BI tool
Cloud	Microsoft Fabric Pipeline	dbt (Fabric Data Build Tool job)	Fabric Lakehouse / Warehouse	Power BI semantic model + report

2. Prerequisites

2.1 Local Solution

- Docker Desktop (or Docker Engine) with Docker Compose installed.
- ~4 GB free RAM for Airflow + Postgres containers.
- Raw course CSVs already produced by the scrapers (coursera_final_data.csv, udacity_final_data.csv, udemy_final_data.csv) placed where dbt seeds expect them.
- Web browser to access the Airflow UI.

2.2 Cloud Solution

- Access to the Microsoft Fabric workspace containing the Notebook, dbt project (Data Build Tool item) and Power BI dataset.
- Permissions to run/trigger the Fabric pipeline named "Pipelining".
- A Power BI account/license to view or edit the published report (PowerBI_Dashboard).

3. Running the Pipeline — Local Solution (For Data Engineers)

3.1 Step 1 — Start the Environment

Open a terminal, navigate to the "local solution/airflow_home" folder, and run:

```
docker compose up -d
```

This starts Postgres (Airflow's metadata DB), runs airflow-init (creates the admin user and initializes the database), then starts the Airflow webserver and scheduler containers.

3.2 Step 2 — Open the Airflow UI

1. Open a browser and go to http://localhost:8080
2. Log in with username "airflow" / password "airflow" (or "admin"/"admin" if using the admin account created by airflow-init).

3.3 Step 3 — Trigger the DAG

3. Locate the DAG named course_data_pipeline.
4. Click the toggle switch next to its name to unpause it.
5. Click the Play (▶) button on the right and select "Trigger DAG".

The DAG executes five tasks in sequence:

Order	Task ID	Purpose
1	dbt_deps	Installs dbt packages (dbt_utils 1.4.0)
2	dbt_seed	Loads the raw CSV files into DuckDB raw tables
3	dbt_snapshot	Captures SCD Type 2 history in offering_snapshot
4	dbt_run	Builds staging → intermediate → marts (dims + fact) → semantic models
5	dbt_test	Executes 73+ automated data-quality tests

3.4 Step 4 — Monitor Progress

- Light green = task currently running.
- Dark green = task finished successfully.
- Red = task failed — click the task box, then "Logs", to see the error (most commonly: raw seed CSVs are missing).

3.5 Step 5 — Query the Results

Once all tasks are dark green, the warehouse file dbt_local.duckdb contains the finished star schema (8 dimension tables + fact_offerings). Data Engineers can connect any DuckDB-compatible client to query it directly; Analysts can point a local BI tool at the same file.

3.6 Step 6 — Shut Down

```
docker compose down -v
```

This stops and removes the containers (the -v flag also removes the Postgres volume; your DuckDB warehouse file on disk is not affected).

3.7 Common Issues

Symptom	Likely Cause	Fix
dbt_seed fails	Raw CSVs not present in the seeds folder	Run the scrapers first, or copy the provided clean-data CSVs into the seeds directory
dbt_test fails on relationships	A dimension lookup did not match (e.g. new instructor not seen before)	Re-run dbt_run then dbt_test; investigate the specific failing model in the logs
Webserver unhealthy / port 8080 busy	Another process is using port 8080	Stop the conflicting process or change the port mapping in docker-compose.yml

4. Running the Pipeline — Cloud Solution (For Data Engineers)

The cloud solution runs the same logical pipeline inside Microsoft Fabric instead of Docker/Airflow.

4.1 Step 1 — Open the Fabric Workspace

Navigate to the Fabric workspace that hosts the "Pipelining" pipeline, the dbt "Data Build Tool" job, and the Power BI dataset.

4.2 Step 2 — Run / Schedule the Pipeline

6. Open the Pipelining pipeline.
7. Click "Run" to execute on demand, or configure a schedule trigger for recurring refreshes.

The pipeline executes three chained activities:

Order	Activity	Type	Purpose
1	Notebook1	TridentNotebook	Ingests/refreshes raw course data into the Lakehouse/Warehouse
2	dbt job1	InvokeDataBuildToolJob	Runs the dbt project (build) to produce staging, intermediate, and mart tables
3	Semantic model refresh1	PBISemanticModelRefresh	Refreshes the Power BI dataset's 9 tables (dims + fact_offerings)

4.3 Step 3 — Monitor the Run

Each activity shows Succeeded/Failed status in the Fabric pipeline run view. Because activities are chained with "dependsOn: Succeeded", a failure stops downstream activities — check the failing activity's output/error details in Fabric monitoring.

4.4 Step 4 — Consume the Dashboard (For Power BI Analysts)

8. Open the PowerBI_Dashboard report in the Power BI service or Power BI Desktop.
9. Confirm the "Last Refreshed" timestamp matches the latest pipeline run.
10. Build or modify visuals using the published semantic model (dim_date, dim_domain, dim_instructor, dim_language, dim_level, dim_offering, dim_offering_type, dim_platform, fact_offerings).
11. To force an on-demand refresh outside the pipeline schedule, use "Refresh now" on the dataset (equivalent to the Semantic model refresh1 activity).

4.5 Common Issues

Symptom	Likely Cause	Fix
Notebook1 fails	Source platform changed page structure, or API auth expired	Check notebook output logs; update scraping/ingestion logic; re-run
dbt job1 fails	A dbt test failed or a model SQL error	Open the Data Build Tool job logs in Fabric; fix the model; re-run the pipeline
Semantic model refresh1 times out	Very large incremental load or capacity throttling	Re-run during off-peak hours or check Fabric capacity metrics

5. Reference Diagrams

The diagrams below illustrate how data and control flow through both deployment modes. Full versions with additional context appear in the Technical Documentation.

5.1 Local Solution — Activity Diagram

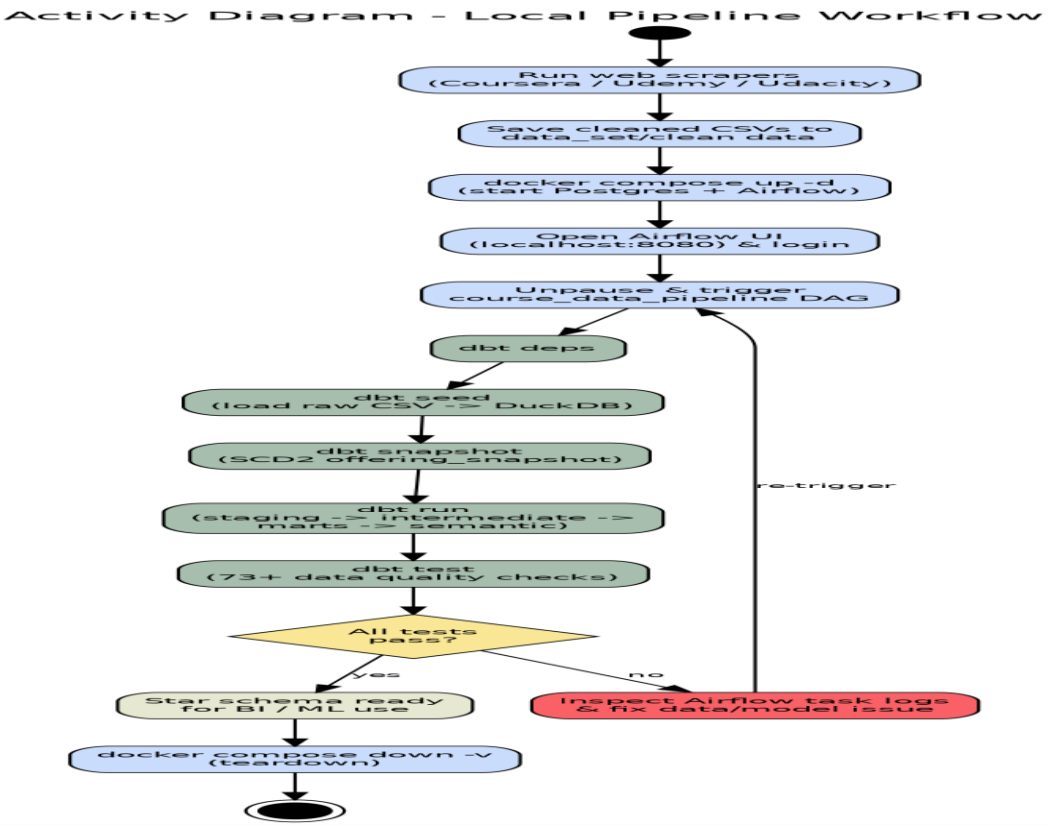

Figure 1. End-to-end activity flow for the Local pipeline.

5.2 Cloud Solution — Activity Diagram

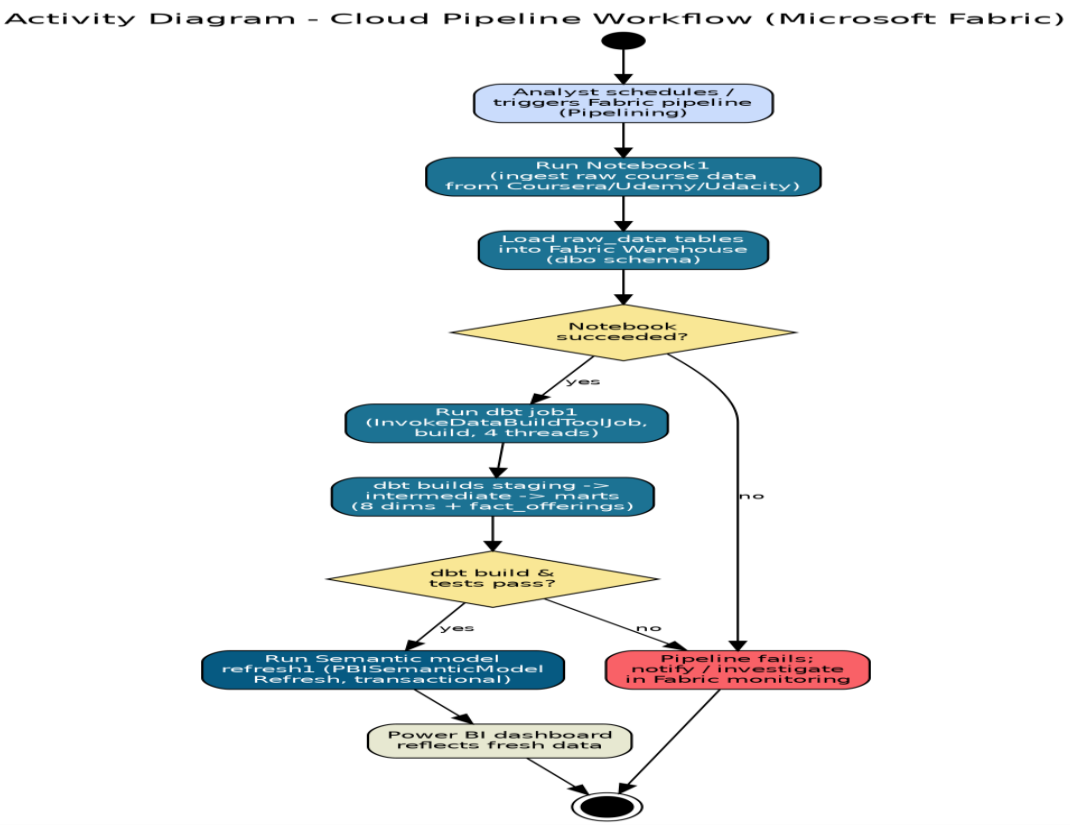

Figure 2. End-to-end activity flow for the Cloud (Fabric) pipeline.

6. Frequently Asked Questions

Q: How often should I trigger the pipeline?

The local DAG is manual-trigger only (schedule_interval=None). The cloud pipeline can be triggered manually or attached to a Fabric schedule trigger for recurring refreshes (e.g., daily).

Q: Where does Slowly Changing Dimension (history) tracking happen?

In both environments, the dbt snapshot "offering_snapshot" tracks changes to title, description, skills, and URL using SCD Type 2 (dbt_valid_from / dbt_valid_to). dim_offering is built directly on top of this snapshot.

Q: Who do I contact if a pipeline run fails repeatedly?

Escalate to the Data Engineering owner of the project (see the Technical Documentation's Support & Ownership section) with the failing task/activity name and the relevant log excerpt.